

NoSQL

Bases de Datos NoSql II. MongoDB

I. C. Juan José Acosta Espinoza

8 octubre 2022

1 Introducción

2 Operaciones CRUD

3 Documentos incrustados

4 Bibliografía

¿Qué es MongoDB?

- MongoDB es un sistema de Bases de Datos NoSQL, orientado a documentos y de código abierto.
- En lugar de guardar los datos en tablas como lo hace el modelo relacional, MongoDB guarda estructuras de datos JSON con un esquema dinámico, haciendo que la integración de los datos en ciertas aplicaciones sea más fácil y rápida.
- El modelo de documentos de MongoDB resulta muy fácil de aprender y usar, y proporciona a los desarrolladores todas las funcionalidades que necesitan para satisfacer los requisitos más complejos a cualquier escala. Se proporcionan drivers para más de diez lenguajes, y la comunidad ha desarrollado varias decenas más.

Las siguientes bibliotecas son oficialmente compatibles con MongoDB. Se mantienen activas, admiten nuevas funciones de MongoDB y reciben correcciones de errores, mejoras de rendimiento y parches de seguridad:

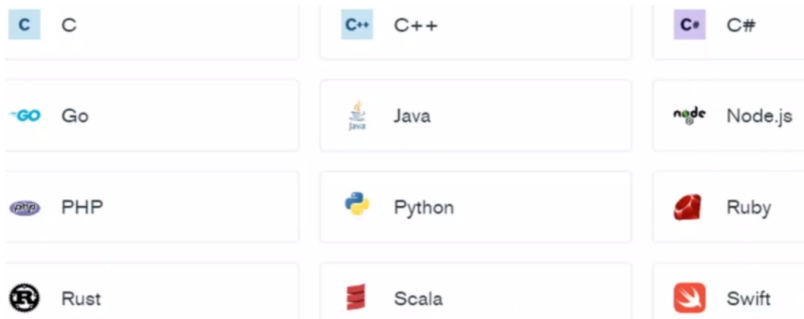


Fig. 1: Lenguajes que tienen un driver para conectarse a MongoDB

- 1 Introducción
- 2 Operaciones CRUD
- 3 Documentos incrustados
- 4 Bibliografía

Operaciones CRUD

MongoDB organiza la información en documentos y estos se agrupan en colecciones. Los documentos tienen la estructura del formato JSON.

comando	uso	Ejemplo
use database	usar una base de datos	use proyecto
db	muestra la base activa	db
show dbs	Enlista las bases de datos	show dbs
show collections	muestra las colecciones de la base de datos activa	show collections
db.coleccion	crea una colección, no hay tipos ni relaciones	db.peliculas
db.coleccion.insert(doc)	inserta un documento en la colección	db.peliculas.insert({name:"Star Wars"})
db.coleccion.drop()	Elimina una colección	db.peliculas.drop()
db.dropDatabase()	Elimina toda la base de datos	db.dropDatabase()

Tabla 1: Comandos básicos de Mongo para la manipulación de bases de datos

Formato JSON

- **JSON** Javascript Object Notation.
- Es un formato para guardar e intercambiar información que cualquier persona pueda leer.
- Los archivos json solo contienen texto y tienen la extensión .json
- Es un formato que almacena información estructurada y se utiliza principalmente para transferir datos entre un servidor y un cliente.
- Hay dos elementos importantes en un objeto json: **claves** (KEYS) y **valores** (values).

- Las keys deben ser cadenas de caracteres (strings).
- Los values son un tipo de dato JSON válido. Puede ser un arreglo, objeto, string, booleano, número o nulo.
- Un objeto JSON comienza y termina con llaves {}. Puede tener dos o más pares de claves:valor dentro , separados por una coma. A si mismo, cada key es seguida de : para distinguirla del valor.

```
{ciudad: "San Luis Potosí", pais: "México", poblacion: 30000, zonas: ["Altiplano", "Huasteca", "Centro", "Media"]}
```

Operaciones CRUD

- Las operaciones de creación o inserción agregan nuevos documentos a una colección. Si la colección no existe, las operaciones de creación también crean la colección.
- Las operaciones pueden afectar uno o varios documentos con un solo comando.
- Se pueden especificar criterios o filtros que identifiquen los documentos a afectar.
- Sintaxis

Basedatos.coleccion.funcionCRUD

función	uso
insertMany ([doc1,doc2,...])	insertar 1 o varios documentos. recibe un arreglo [] de documentos json{}
find()	consulta todos los documentos de la colección
find(filtro)	consulta los documentos que cumplen las condiciones del filtro. El filtro es un objeto json.
find(filtro,campos)	Campos json que indica que campos del documento queremos consultar 1 u ocultar 0 A esto se le llama proyección en MongoDB
find().pretty()	Mejora la presentación de los datos

Tabla 2: funciones CRUD: crear y leer

Al insertar documentos MongoDB les asigna un id único de forma predeterminada

función	uso
<code>replaceOne(filtro,newdoc)</code>	Reemplaza el documento que cumple con las condiciones del filtro por el nuevo documento
<code>replaceOne(filtro,newdoc, upsert:true)</code>	si no existe agrégalo
<code>update(filtro,newdoc)</code>	Reemplaza el documento
<code>update(filtro, { \$set:campos}, { multi:true})</code>	Actualiza o agrega campos a los documentos que cumplen las condiciones Los operadores se identifican por \$
<code>remove(filtro)</code>	Elimina los documentos que cumplen con las condiciones

Tabla 3: funciones CRUD: actualizar, reemplazar y eliminar

***NOTA:** `replaceOne` no tiene equivalente en SQL porque este permite modificar la estructura agregando nuevas propiedades al documento o eliminarlas.

Un cursor es un objeto que sirve para ir iterando sobre el mismo. Es el resultado de ejecutar `find`. Al cursor se le pueden aplicar operaciones.

función	uso
<code>find().count()</code>	Cuenta el número de documentos
<code>find().limit(n)</code>	Limita el número de resultados (n)

Tabla 4: funciones que se pueden aplicar al cursor de `find`

Se puede acceder a los elementos del cursor como si fuera un arreglo (por los índices)

`db.coleccion.find()[i][\"namecampo\"]`

Acceso al campo `namecampo` del `i`-ésimo elemento del resultado de la consulta `find`.

- 1 Introducción
- 2 Operaciones CRUD
- 3 Documentos incrustados**
- 4 Bibliografía

Documentos incrustados en matrices

- Es muy útil incrustar documentos en matrices para poder asociar múltiples documentos a una misma llave
- Esto no es posible en SQL.
- Los documentos incrustados siempre van dentro de una matriz
[]

```
db.coleccion.insert(..., clave : [doc1, doc2, doc3, ...], ...)
```

- Para buscar en claves internas siempre debemos usar "" indicando la clave externa y la clave interna

```
db.coleccion.find("claveext.claveint" : valor)
```

- Si queremos que busque solo en un documento incrustado específico podemos indicar el índice después de la clave externa.

db.coleccion.find("claveext.index.claveint" : valor)

***NOTA:** Hay que tener cuidado de aplicar las operaciones ya conocidas porque si algún documento de la matriz cumple las condiciones de búsqueda find nos regresa todo el documento contenedor

Documento, arreglo y arreglo de documentos

El valor de una clave puede ser:

- **Arreglo** si los elementos a guardar tienen poca relevancia o no importa el orden en el que se consulten.
Ejemplo: comentarios, hashtags, ingredientes, etc.
- **Documento** si cada elemento a almacenar tiene importancia pero solo guarda 1 valor.
Ejemplo: medidas(alto, ancho, profundidad), dirección (colonia, calle, número, etc.), estudios (lic, maestría, doctorado)
- **Arreglo de documentos** si cada elemento a almacenar tiene información propia.

Por ejemplo suponga que quiere almacenar la información de una póliza de seguro.

```
{
  cliente:"Homero J. Simpson",
  edad:37,
  domicilio:{
    estado:"Springfield",
    calle:"Avenida Siempreviva",
    numero:742
  },
  hobbies:["bolos","beber","ver tv"],
  asegurados:[
    {nombre:"Marge",edad:34,parentesco:"esposa"},
    {nombre:"Bart",edad:12,parentesco:"hijo"},
    {nombre:"Lisa",edad:8, parentesco:"hija"},
    {nombre:"Maggie",edad:1, parentesco:"hija"}
  ]
}
```

Documento 1: Documento de la colección clientes

- 1 Introducción
- 2 Operaciones CRUD
- 3 Documentos incrustados
- 4 Bibliografía

Bibliografía

- Manual de MongoDB
- Formato JSON